Hackathon-Anti-Fraud-Pix

Integrantes: Pedro H. S. Prestes · Laura Pazini Medeiros

Um motor antifraude de Pix: uma API que recebe cada Pix antes de ele sair e responde se ele pode seguir (`approved`), se o cliente precisa confirmar com um segundo fator (`two_factor_required`) ou se deve ser barrado (`rejected`). Toda resposta vem com o motivo, em português. O projeto nasceu do `bootcamp-base-api` e usa Python 3.11, FastAPI, PostgreSQL 16 e Docker. O enunciado completo [1] está em docs/01-contexto/enunciado.md.

Em resumo

- **15 regras explícitas decidem**, cada uma com um motivo que um atendente consegue ler para o cliente.
- **5 modelos de machine learning rodam ao lado, em modo sombra**: calculam e registram tudo, mas não decidem, porque não temos rótulos reais de fraude.
- **As regras foram calibradas e testadas num simulador de terceiros** (o PixSim [5], calibrado com dados abertos do Banco Central), e não só no nosso, que seria um teste viciado.
- **113 testes automatizados** exercitam o motor pela própria API, e o CI está verde.

Como o motor de decisão funciona

Quando um Pix chega, a API primeiro confere se ele já foi analisado (o mesmo Pix enviado duas vezes recebe a mesma resposta). Depois o motor busca no banco o histórico que importa: o que esse pagador costuma fazer, se o aparelho é conhecido, quem é o destinatário e há quanto tempo a conta dele existe. Com isso, **todas as regras são avaliadas** e vence a decisão mais severa. O Pix, a análise e os motivos são gravados juntos, e qualquer análise pode ser consultada depois pelo `id` do Pix.

O trabalho se divide em dois caminhos, como num banco de verdade:

- **Caminho rápido**, dentro da requisição: as regras decidem em milissegundos, e os modelos em sombra dão a opinião deles.
- **Caminho de fundo**, num serviço separado (`worker`): a cada 30 segundos, ele olha a rede de quem paga quem e calcula a reputação de cada conta de destino, o que ajuda a reconhecer contas laranja. O caminho rápido só lê esse resultado pronto.

Os detalhes estão em motor de regras, nos diagramas C4 e nas decisões de arquitetura.

Regras

Como escolhemos. Uma regra entra no motor se tiver valor por pelo menos um destes caminhos:

1. **Pega fraude no simulador independente** e paga o atrito que causa nos clientes bons.
2. **Segue uma norma do Banco Central**, mesmo que o simulador preferisse outra coisa.
3. **Cobre um golpe que o simulador não sabe gerar**: lá ela não dispara e não custa nada, e aqui ela protege contra o golpe real.

Os valores marcados com ★ foram ajustados no simulador, escolhidos num cenário e confirmados em outro. Os demais seguem norma ou são escolhas do time, sinalizadas como tal. Todos podem ser trocados por variável de ambiente (`RULE_*` em `src/constants.py`). O porquê completo de cada número, com fontes e medições, está no catálogo de regras.

As regras que mais protegem

São as que, no simulador, mais barram golpes por unidade de incômodo ao cliente bom.

Regra	Lógica	Parâmetros	Por quê
`COUNTERPARTY_FIRST_TIME_HIGH`	Primeiro Pix do pagador para esse destinatário, com valor alto	★ acima de R$ 1.000	Quase todo golpe manda o dinheiro para alguém com quem a vítima nunca transacionou: o falso parente, o falso boleto, o falso funcionário do banco. Já o aluguel vai sempre para o mesmo lugar. O Banco Central lista a recorrência entre pagador e recebedor como fator de risco (Regulamento do Pix, art. 39-B [12]). É, de longe, a regra que mais protege
`PROFILE_NEW_DEVICE`	Pagador com histórico usando um aparelho que nunca usou	★ a partir de R$ 200	Trocar de celular é o primeiro passo de quem tomou a conta de alguém [4]. O piso de valor evita incomodar quem só pagou um café do celular novo

Regra	Lógica	Parâmetros	Por quê
VELOCITY_POISSON_GAMMA	Rajada de Pix na última hora, improvável para o ritmo daquele pagador	★ chance abaixo de 30 em um milhão; exige 24 h de histórico	"Rápido demais" é diferente para cada pessoa. Em vez de um número fixo, comparamos a pressa de agora com o histórico do próprio cliente, como pede a literatura de detecção de fraude (Bolton & Hand [3]; modelo Gamma–Poisson [19]). Pega a vítima pressionada fazendo vários Pix seguidos
TYPOLOGY_ACCOUNT_TAKEOVER	Pagador com histórico + aparelho novo + (localização impossível ou valor muito fora do padrão)	reprova se o valor estiver mais de 3 desvios acima do normal	Cada sinal sozinho pede só uma confirmação; juntos, descrevem uma conta tomada por terceiros [4], e aí o Pix é barrado
PROFILE_NEW_SOURCE_ACCOUNT	Conta pagadora recém-aberta movimentando valor alto	menos de 7 dias e acima de R$ 2.000	Conta aberta com documentos falsos costuma ser usada logo. Raro, mas quando dispara é quase sempre fraude

Regras para golpes que o simulador não gera

O simulador não tem geolocalização, usa um aparelho por conta e espalha as vítimas no tempo. Estas regras ficam caladas lá, então não custam nada, e cobrem golpes conhecidos.

Regra	Lógica	Parâmetros	Por quê
VELOCITY_SOURCE_COUNT	Tentativas do pagador em 10 min, contando as recusadas	2FA a partir de 5; reprova a partir de 10	Conta tomada é esvaziada em rajada, e insistir depois de uma recusa também é sinal. Protege até quem não tem histórico. Limites escolhidos pelo time
VELOCITY_DEVICE_SOURCES	Contas diferentes pagando pelo mesmo aparelho em 24 h	reprova acima de 3	Um celular operando muitas contas é uma "fazenda" de contas ou um golpista com credenciais de várias vítimas. Uma família no mesmo aparelho raramente passa de 3
PROFILE_GEO_VELOCITY	Distância até o último Pix com GPS dividida pelo tempo entre eles	mais de 50 km a mais de 900 km/h	Ninguém vai de São Paulo ao Rio em dez minutos. É sessão em outro aparelho ou localização falsificada. 900 km/h é a velocidade de um avião comercial
COUNTERPARTY_FAN_IN	Pagadores diferentes mandando para o mesmo destino em 1 h	reprova acima de 10	Conta que recebe de muita gente ao mesmo tempo funciona como funil de golpe
COUNTERPARTY_GRAPH_RISK	Reputação da conta de destino, calculada pelo serviço de fundo a partir da rede de pagamentos	2FA a partir de 0,6; reprova a partir de 0,9	Conta laranja se reconhece pela rede: recebe de muitos, repassa logo, é nova. O serviço de fundo junta esses sinais num score

Mantidas por regulação ou cobertura

No simulador, estas regras custam um pouco mais do que evitam: incomodam clientes bons sem pegar golpes que ele não gera. Ficam por um motivo que o simulador não mede.

Regra	Lógica	Parâmetros	Por quê
LIMIT_NIGHT_AMOUNT	Pix de pessoa física entre 20h e 6h acima do teto	acima de R$ 1.000	É o limite noturno que o Banco Central manda os bancos aplicarem (Res. BCB 142/2021, IN BCB 512/2024 [11]), por causa do sequestro relâmpago [4]. Pedimos 2FA em vez de barrar, porque o cliente pode ter pedido limite maior
COUNTERPARTY_NEW_ACCOUNT	Idade da conta que recebe	2FA se tiver menos de 30 dias; reprova se tiver menos de 3 dias e o Pix passar de R$ 1.000	O Banco Central manda considerar há quanto tempo a conta do recebedor existe (art. 39-B [12]), porque conta laranja é aberta para o golpe. No simulador as mulas são contas antigas, por isso a regra parece não ajudar lá
TYPOLOGY_MULE	Conta nova recebendo de muitos pagadores em 1 h	menos de 30 dias e 5 ou mais pagadores	Combina os dois sinais clássicos de conta laranja e barra antes do funil genérico
PROFILE_AMOUNT_ROBUST_Z	Valor muito acima do padrão do próprio pagador nos últimos 30 dias	★ mais de 4,5 desvios; exige 5 Pix de histórico	Valor fora do normal para aquela pessoa, medido com estatística robusta [13] (mediana, não média, para um único Pix grande não distorcer o perfil)
LIMIT_SINGLE_AMOUNT	Valor de um único Pix acima do teto	R$ 50 mil (CPF); R$ 200 mil (CNPJ)	Teto de exposição do banco num Pix só; o Regulamento do Pix (art. 37 [12]) pede critério de risco, sem fixar valor. É a única regra que cobre "valor alto demais" para qualquer cliente

Descartadas. Três regras saíram do motor depois de medidas: fracionamento em vários Pix pequenos, destino que já teve Pix recusado e soma diária. Elas incomodavam mais clientes bons do que golpes evitavam, nos dois cenários do simulador, e nenhuma tinha base regulatória. O motivo e os números estão na ADR 0033.

Como as regras viram decisão

Todas as regras são avaliadas, sem parar na primeira, e vence a mais severa: **barrar > pedir 2FA > aprovar**. Se nenhuma disparar, o Pix é aprovado.

- **Pedimos 2FA** quando um sinal isolado gera dúvida, mas também acontece com cliente bom: valor acima de um limite, aparelho novo, valor fora do padrão, conta de destino nova, primeiro Pix alto para alguém. O cliente legítimo resolve com um toque.
- **Barramos** quando o risco é claro: um sinal extremo (conta de destino com menos de 3 dias recebendo mais de R$ 1.000, 10 Pix em 10 minutos, aparelho com mais de 3 contas, mais de 10 pagadores para o mesmo destino em 1 hora) ou uma combinação que descreve um golpe conhecido (conta tomada, conta laranja). Duas dúvidas não somam um bloqueio: só uma combinação explícita bloqueia.
- A resposta lista **todos** os motivos, do mais grave ao mais leve, com os números que cada regra usou e a versão do conjunto de regras.

Exemplos em como as regras viram decisão.

Como validamos

O nosso primeiro simulador gerava os golpes que nós mesmos desenhamos, e as regras foram escritas para pegá-los. Medir nele seria como corrigir a própria prova. Por isso todas as métricas vêm do **PixSim** [5], um simulador aberto, feito por outros autores e calibrado com dados públicos do Banco Central, com golpes que não desenhamos: engenharia social, conta tomada, Pix errado e QR adulterado.

- **Cliente bom quase não é incomodado:** cerca de 3% dos Pix legítimos pedem 2FA e praticamente nenhum é barrado.
- **Conta tomada é quase sempre pega:** em 99% dos casos, algum Pix do golpe é barrado.
- **Engenharia social é o ponto fraco:** quando a própria vítima, enganada, faz o Pix do celular dela, o motor só consegue pedir 2FA, e a vítima pressionada pode confirmar. Esse é o limite mais honesto do projeto, discutido no relatório.
- **Comparado a não ter motor**, o prejuízo esperado cai cerca de 20%, já contando o atrito com os clientes bons.

Os números completos estão na validação no PixSim e na calibração. O nosso simulador continua no projeto só como teste automático: garante que nenhuma mudança quebre o que já funcionava.

Além das regras: modelos em sombra

Rodamos cinco modelos em todo Pix: regressão logística calibrada [14], LightGBM [15] e Isolation Forest [16] no caminho rápido; GraphSAGE [17] (rede de pagamentos) e HMM [18] (sequência de Pix) no serviço de fundo. Eles calculam um score e registram o que **decidiriam**, mas não decidem. Assim o banco compara o modelo com as regras no mesmo tráfego, sem arriscar o cliente.

Por que não deixá-los decidir? Os rótulos de fraude que temos são sintéticos. No simulador, o modelo junto com as regras barraria bem mais golpes, mas só porque treino e teste vêm do mesmo gerador. Quando pioramos o modelo até a qualidade típica de dados reais, a vantagem some. Ligar o modelo é uma linha de código; o que falta é rótulo real, que viria das contestações do MED, com dias de atraso [7]. Mostramos como esse ciclo funcionaria num backtest, descrito no relatório.

Os modelos são treinados fora da API e servidos a partir de arquivos JSON, sem bibliotecas de machine learning na imagem da `api`. Detalhes nas ADRs 0022 a 0026 e na validação.

Como rodar e testar

Pré-requisitos: Docker com `docker compose`, Git e Python 3.11 (ou `uv`).

```
make all      # sobe api, db e worker, cria o .venv (Python 3.11) e roda os testes
```

Sem `make`:

```
docker compose up -d --wait
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -v
```

A API responde em `http://localhost:3000`. Toda rota, menos `/` e `/health_check`, exige o cabeçalho `INTERNAL-TOKEN: default_token`.

```
curl -X POST http://localhost:3000/pix/transaction \
  -H "INTERNAL-TOKEN: default_token" -H "Content-Type: application/json" \
  -d @docs/03-api/exemplo-pix.json

curl http://localhost:3000/pix/transaction/pix-000123 -H "INTERNAL-TOKEN: default_token"
```

Ferramentas de análise (rodam fora da API):

Comando	Para quê
`make backtest`	Roda o mesmo motor sobre um arquivo de Pix, sem API e sem banco
`make parity`	Confere que a API e o backtest decidem igual, Pix a Pix
`make calibrate-pixsim`	Escolhe os parâmetros no simulador independente
`make rule-ablation`	Mede quanto cada regra contribui, desligando uma de cada vez
`make calibrate-check`	Trava do CI: impede que uma mudança aumente o incômodo ao cliente ou deixe golpe passar
`make train-models`	Retreina os modelos em sombra

Mais comandos em rodando localmente.

Documentação

Quero entender…	Leia
o que foi pedido	enunciado
por que cada decisão	decisões (ADRs)
cada regra, com fontes e números	catálogo de regras
o contrato da API	rotas e erros
o banco	modelo de dados
como testamos	estratégia de testes
o que mudou	CHANGELOG
o índice completo	docs/README.md

Agentes de código seguem o `AGENTS.md`.

Referências

[1] QI Tech. *Desafio Antifraude Pix* e slides de abertura do Hackathon QI Tech 2026 (Risk Solutions). Transcrição em `docs/01-contexto/enunciado.md`; PDFs em `docs/referencia/enunciado/`.

[2] Pesquisa QITech (documento interno do time, literatura branca e cinzenta). `docs/pesquisa/Pesquisa QITech.pdf`.

[3] Bolton, R. J.; Hand, D. J. (2002). *Statistical Fraud Detection: A Review*. Statistical Science, vol. 17.

[4] Pizzolato et al. (2025). *A Taxonomy of Pix Fraud in Brazil*. arXiv:2511.20902. https://arxiv.org/abs/2511.20902

[5] Zeimarani, B.; Khatib, A.; Mousavinasr, S.; Figueiredo, C. M. S. (2026). *PixSim: a calibrated open-source simulator of instant-payment fraud, recovery and interdiction under analyst capacity constraints*. Identificador arXiv ainda não publicado (TODO no `CITATION.cff` do repositório; não verificado). Código: https://github.com/BashirZeimarani/pixsim_public (Apache-2.0; commit `7fdfc90013960f46df60c3ea01838e768e7a420a`).

[6] Dal Pozzolo, A.; Caelen, O.; Johnson, R. A.; Bontempi, G. (2015). *Calibrating Probability with Undersampling for Unbalanced Classification*. IEEE SSCI 2015.

[7] Dal Pozzolo, A. et al. (2018). *Credit card fraud detection: a realistic modeling and a novel learning strategy*. IEEE Transactions on Neural Networks and Learning Systems, 29(8).

[8] Dal Pozzolo, A. et al. (2014). *Learned lessons in credit card fraud detection from a practitioner perspective*. Expert Systems with Applications, 41(10).

[9] Lucas, Y. et al. (2019). *Multiple perspectives HMM-based feature engineering for credit card fraud detection*. ACM SAC 2019.

[10] Carcillo, F. et al. (2018). *Scarff: a scalable framework for streaming credit card fraud detection with Spark*. Information Fusion 41, 182-194. doi:10.1016/j.inffus.2017.09.005

[11] Banco Central do Brasil. Resolução BCB nº 142/2021, art. 2º, I (limite noturno de R$ 1.000, 20h às 6h, pessoa física), hoje consolidado na Instrução Normativa BCB nº 512/2024, art. 3º, §3º e §7º, II.

[12] Banco Central do Brasil. Regulamento do Pix, arts. 37 e 39-B, §1º, I a V (fatores de avaliação de suspeita, como tempo desde a abertura da conta e recorrência; o 39-B trata do PSP do recebedor).

[13] Iglewicz, B.; Hoaglin, D. C. (1993). *How to Detect and Handle Outliers*. ASQC Quality Press (escore z modificado com mediana e MAD; resumo no NIST/SEMATECH e-Handbook, seção 1.3.5.17).

[14] Platt, J. C. (1999). *Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods*. Em *Advances in Large Margin Classifiers*, 61-74.

[15] Ke, G. et al. (2017). *LightGBM: a highly efficient gradient boosting decision tree*. NeurIPS 30, 3146-3154.

[16] Liu, F. T.; Ting, K. M.; Zhou, Z.-H. (2008). *Isolation forest*. IEEE ICDM, 413-422. doi:10.1109/ICDM.2008.17

[17] Hamilton, W. L.; Ying, R.; Leskovec, J. (2017). *Inductive representation learning on large graphs*. NeurIPS 30, 1024-1034.

[18] Rabiner, L. R. (1989). *A tutorial on hidden Markov models and selected applications in speech recognition*. Proceedings of the IEEE 77(2), 257-286.

[19] Gelman, A.; Carlin, J. B.; Stern, H. S.; Dunson, D. B.; Vehtari, A.; Rubin, D. B. (2013). *Bayesian Data Analysis*, 3ª ed. CRC Press.

Licença

MIT